DrivePulse AI

Machine Learning Development and Implementation Guide

A beginner-friendly account of the project from dataset selection to the Step 2B mobile application

Repository snapshot reviewed: **17 October 2026**

Primary vehicle dataset: **2018 Nissan Leaf field telemetry**

ML experiment: **Isolation Forest `0c777345c64dfeba`**

Preparation run: **`1d6fc5b045ee25d0`**

> **Scope and honesty statement**
>
> This guide explains what the repository actually contains. DrivePulse AI currently replays two short, genuine recordings and their previously calculated anomaly scores in a mobile application. It does not yet run the model through FastAPI, diagnose battery faults, estimate failure probability, predict remaining useful life, or prove performance on other vehicles.

How to use this guide

Read Chapters 1–2 for the product idea and basic ML concepts. Chapters 3–9 follow the data and model work in order. Chapters 10–13 explain how verified results became a mobile replay. Chapters 14–15 separate the current architecture from future work. Chapter 16 is a reading map for the repository.

Evidence labels used in this report

Label	Meaning
Verified result	A number or behavior recorded in a report, manifest, saved artifact, or source file.
Implemented	Code for the behavior exists in the current repository.
Tested	An automated check or documented build/export check passed.
Planned	The repository or project brief describes the work, but the current implementation does not provide it.
Unknown	The available evidence does not establish the answer.

Table of contents

1. Introduction to DrivePulse AI
2. Understanding the Machine Learning Problem
3. Dataset Selection and ML Readiness
4. Exploratory Data Analysis
5. Data Preparation and Preprocessing
6. Isolation Forest Model Training
7. Anomaly Scores and Threshold Selection
8. Model Evaluation and Validation
9. Problems Encountered and How We Addressed Them
10. Selection of the Two Demo Recordings
11. How ML Results Reached the Mobile App
12. Step 2A: Mobile Architecture and Implementation
13. Step 2B: Professional UI Improvements
14. Current Project Architecture and Status
15. Future Development
16. Important Files and Their Purposes

1. Introduction to DrivePulse AI

1.1 The product idea

DrivePulse AI began as a vehicle-health monitoring and diagnostic application. The broad vision is to collect vehicle telemetry, recognize unusual behavior, predict failures, estimate remaining useful life, and present understandable insights and maintenance reports. That vision is useful because a modern vehicle produces many measurements over time. A driver cannot reasonably inspect every voltage, current, temperature, speed, and status value by hand.

Machine learning can help summarize patterns in those measurements. It may notice combinations that are rare compared with earlier observations. With the right labeled history, a later system might learn links between sensor behavior and confirmed component failures. The words **with the right labeled history** matter: an algorithm cannot learn verified fault categories or remaining useful life from a dataset that has no verified faults, repair events, or end-of-life outcomes.

The current MVP therefore has a narrower purpose. It demonstrates an evidence-preserving path from real recorded telemetry to an experimental anomaly score and then to a mobile interface. It is a learning and integration prototype, not a vehicle diagnostic product.

1.2 Product vision versus current MVP

Capability	Complete vision	Current repository
Vehicle data	Live or uploaded telemetry from vehicles	Two short exported recordings from one 2018 Nissan Leaf
ML behavior	Fault detection, prediction, health estimation	One Isolation Forest that finds statistically unusual combinations of three battery measurements
Inference	Backend scores incoming observations	Scores were calculated offline and saved; runtime inference is not connected
Mobile experience	Live dashboard and actionable diagnostics	Deterministic playback of saved measurements and scores
Evidence	Labeled failures and independent vehicles	No verified fault labels; one recorded vehicle
Persistence	Full user, vehicle, telemetry, and report storage	Firebase authentication and a minimal PostgreSQL profile design; recorded replay is local and independent of PostgreSQL

This difference is intentional. The MVP proves that the project can preserve provenance, prepare time-series data without obvious leakage, train a reproducible baseline, and communicate the result honestly.

1.3 Why the 2018 Nissan Leaf was selected

The repository uses a public research dataset published by the UbiLab group at Macao Polytechnic University. It contains field telemetry from a 2018 Nissan Leaf and includes high-voltage battery measurements, battery temperatures, state of charge, drivetrain signals, timestamps, and trip identifiers. This makes it more relevant to electric-vehicle monitoring than a generic tabular fault dataset.

The download also includes a parameter-description workbook and both raw and researcher-preprocessed CSV files. That combination allowed the project to compare source telemetry with a processed version and to check the meaning

and units of many fields. However, public download access is not an explicit open-data license. The source asks researchers to cite the associated work, and redistribution rights remain unresolved.

1.4 What has actually been completed

The following work is present and documented:

1. The official raw and preprocessed files and parameter workbook were downloaded, hashed, and recorded in a manifest.
2. Reproducible EDA measured schemas, missingness, distributions, duplicate behavior, timestamps, trip structure, formula consistency, and other quality concerns.
3. ML-readiness analysis evaluated anomaly detection, fault classification, battery health estimation, and predictive maintenance against the evidence actually present.
4. A leakage-aware preparation pipeline created a separately versioned three-feature battery view.
5. An Isolation Forest baseline was trained and saved with a frozen score threshold and interval-grouping protocol.
6. Two validation clips were selected and exported with genuine source rows, measurements, scores, and interval metadata.
7. A React Native/Expo application replays those clips through Overview, Telemetry, Insights, Replay, and Insight Details screens.
8. Step 2B added a consistent visual system, clearer charts, replay controls, and accessibility metadata. Automated TypeScript, lint, Jest, and Android JavaScript export checks passed; physical-device visual acceptance remains outstanding.

2. Understanding the Machine Learning Problem

2.1 What problem did we want ML to solve?

The practical question was: **Can we automatically identify battery telemetry that looks unusual compared with the vehicle's observed history?**

Suppose a vehicle usually reports voltage, temperature, and SOC combinations inside familiar regions. A new combination might sit far away from those regions. An anomaly detector can assign it a more unusual score and draw attention to it. This is useful for review, but it does not explain the cause.

For example, a temperature of 40 °C may be normal during one operating condition and unusual during another. A rare combination could reflect hard driving, charging, weather, a sensor that has stopped refreshing, data-acquisition trouble, or a real physical issue. The algorithm only sees numbers.

2.2 Supervised and unsupervised learning

Supervised learning uses examples paired with correct answers. For fault classification, an example might include telemetry plus a label such as `battery_cooling_failure`. The model learns a mapping from measurements to those labels. A trustworthy evaluation then checks predictions against confirmed labels that were hidden from training.

Unsupervised learning has no correct-answer label for each row. It tries to find structure in the input values themselves. An anomaly detector can learn which combinations are common or rare, but it cannot know whether a rare combination is mechanically dangerous.

The Nissan Leaf files contain no verified healthy/faulty labels, no confirmed fault categories, no diagnostic trouble-code outcomes, and no linked repair records. That prevents an evidence-based supervised fault classifier. Unsupervised anomaly detection was selected because it can support a controlled experiment without inventing labels.

2.3 Why start with Isolation Forest?

Isolation Forest is a lightweight unsupervised algorithm available in scikit-learn. It works with numerical feature tables, handles large datasets efficiently, and produces a continuous score for every observation. Those properties made it suitable for a first baseline with more than 1.6 million prepared rows and only three input features.

It also creates a useful engineering baseline. Before attempting a neural network or a temporal autoencoder, the team can learn whether the data, score distribution, threshold policy, alert aggregation, provenance, and mobile presentation work end to end.

Isolation Forest can detect unusual points relative to its training distribution. It cannot by itself:

- identify a mechanical cause;
- estimate a probability of failure;
- prove that an observation is unsafe;
- learn a time sequence merely because timestamps exist;
- generalize from one vehicle to every Nissan Leaf;
- replace diagnostic confirmation by a technician or a validated fault label.

2.4 The important language distinction

The mobile application uses **typical** and **statistically unusual**. It avoids phrases such as “battery failure detected,” “critical damage,” or “92% probability of failure.” This is scientifically important, not just cautious wording.

An anomaly score answers:

> How unusual is this three-number combination relative to patterns isolated by the trained model?

It does not answer:

> Is the battery broken, what caused the issue, and when will it fail?

3. Dataset Selection and ML Readiness

3.1 Source and provenance

The primary source is the UbiLab dataset page hosted by Macao Polytechnic University. The repository records these official downloads:

File	Role	Size	SHA-256 prefix
Dataset_NissanLeaf-2021_2022.csv	Original raw download	1,538,549,900 bytes	7fe9682e6132…
Dataset_Preprocessed.csv	Researcher's preprocessed download	443,747,562 bytes	609efebf1004…
Nissan_Leaf_Model2018_Data_Description.xlsx	Parameter descriptions	28,139 bytes	3e2d4d44fbb0…

The repository manifest retains the direct URLs, HTTP metadata, exact sizes, hashes, and acquisition status. No semantic dataset version was published, so the file checksum identifies the snapshot. No explicit dataset license or commercial redistribution permission was found on the source page or workbook.

The publisher describes a 2018 Nissan Leaf and nominal 250 ms logging. The actual downloaded files, not the publisher's broader sample-count statement, determine the measured dimensions below.

3.2 Actual file structure

Property	Raw	Preprocessed
Rows	2,322,105	1,918,299
Columns	92	32
Distinct trip IDs	227	127
Recorded days	121	82
Time coverage	12 Nov 2020–3 Aug 2022	12 Nov 2020–8 Jul 2022
Verified label columns	0	0
Missing cells	11,595,760	0
File compression	None	None

The raw file has measurement columns, categorical status fields, timestamps, export indices, and derived columns. It includes speed, gear, RPM, SOC, SOH, torque, motor temperature, tire pressures, charge indicators, high-voltage battery voltage and current, multiple pack temperatures, cell extrema, remaining energy, power-related values, trip identifiers, accumulated values, and calendar encodings.

The preprocessed file keeps 32 columns. It is smaller and has no missing cells, but it is not an independent dataset. The two files share 127 trip IDs, and the analysis found 1,886,696 exact overlapping timestamps. Pooling them would duplicate related observations and create misleading sample counts.

3.3 Columns are not automatically model features

A **column** is anything stored in the table. A **model feature** is a value intentionally supplied to the algorithm. Timestamps, trip IDs, export indices, and labels may be columns without being valid model inputs.

This project treated identifiers and timestamps as metadata. They are essential for chronological splitting, source-row tracing, and interval grouping, but the Isolation Forest did not receive them as ordinary numbers. Feeding a trip number or epoch timestamp directly could let a model memorize when a row was recorded instead of learning battery behavior.

3.4 ML-readiness findings

Readiness means asking whether the available evidence can support a meaningful experiment and evaluation. It is more demanding than asking whether a CSV can be loaded.

Proposed task	Repository assessment	Why
Anomaly detection	**Conditionally ready—further verification required**	Unusual patterns can be studied, but no verified normal/fault reference exists.
Fault classification	**Not ready—essential evidence missing**	No confirmed fault categories, DTC outcomes, normal controls, or event times.
Battery health estimation	**Conditionally ready—further verification required**	Reported BMS SOH can be monitored; it is not independent measured capacity, and one vehicle cannot support general claims.
Predictive maintenance	**Not ready—essential evidence missing**	No linked failure, service, replacement, or end-of-life outcomes.

Energy-consumption target construction was also blocked in preparation because current polarity, charging/regeneration identification, and independently verified energy accounting remained uncertain. Battery-temperature forecasting was judged conditional: safe sequence indices exist, but target semantics and independent validation remain limited.

3.5 The SOH shortcut problem

In the preprocessed data, SOH ranged from 92.63% to 96.46%, had only 56 unique values, and stayed equal across 99.98% of adjacent within-trip rows. Only two processed trips contained within-trip SOH changes. SOH was strongly associated with odometer and recording time; the reported SOH–odometer correlation was approximately −0.95081.

This does not prove that mileage alone explains battery degradation. It shows a major modeling risk: because the data comes from one vehicle moving forward through time, mileage, calendar time, and aging are entangled. A model might memorize that vehicle's timeline instead of learning a general relationship between physical battery stress and health. This is why the repository does not present a general SOH predictor.

4. Exploratory Data Analysis

4.1 What EDA means

Exploratory Data Analysis, or EDA, is the structured examination of a dataset before modeling. It asks questions such as:

- What does one row represent?
- Which columns are measurements, identifiers, statuses, or derived values?
- Are values missing, repeated, impossible, or poorly documented?
- How are timestamps spaced?
- Do later dates behave like earlier dates?
- Are two columns effectively copies of the same signal?

Without EDA, a model can appear accurate while learning a timestamp shortcut, a missing-value code, duplicated observations, or a formula derived from its target.

4.2 What this project actually inspected

The EDA scripts produced machine-readable tables and figures for dimensions, data types, semantic roles, missingness, duplicates, descriptive statistics, distributions, correlations, time coverage, sampling intervals, trip continuity, formula checks, and vehicle/battery measurements. The parameter workbook was extracted into a dictionary and cross-referenced with actual columns.

The EDA found no exact full-row duplicates and no duplicate timestamp or trip-timestamp rows in either file. It did find repeated measurement vectors: 13,733 in raw data and 35,953 in preprocessed data when identifiers, times, and derived fields were excluded. Repeated telemetry can be legitimate when a sensor value stays unchanged, so those rows were investigated rather than automatically deleted.

4.3 Missing and unavailable measurements

The raw file contains 11,595,760 missing cells. `SWA` and `Pack_3_C_Temp` are entirely empty. Raw SOH has 2,247 missing readings and 2,853 zero readings. By contrast, the researcher-preprocessed file has no missing cells, but absence of null values does not guarantee that every value is meaningful.

One especially important pattern was `Speed = -1`:

- 266,235 raw rows have negative speed;
- every negative value is exactly −1;
- 266,214 of those rows also have `RPM = -1` and `Torque = -1`;
- in the test period, 182,885 of 182,887 negative-speed rows share those drivetrain values.

Only a small fraction coincided with Reverse gear. This strongly suggests unavailable drivetrain telemetry, although the workbook does not explicitly define −1 as the sentinel for these fields. The project therefore labels the pattern as **suspected unavailable telemetry**, not as confirmed reverse driving or a verified fault.

4.4 Outliers and suspicious values

EDA distinguishes a statistical outlier from an invalid value. A high battery temperature can be rare but physically real. A field can also show an acquisition artifact without representing a vehicle fault.

The raw audit flagged 2,141 high-voltage readings for review using a documented analysis threshold, 3,850 pack-temperature sentinel-band rows, 457,168 non-positive tire-pressure rows, 202 odometer decreases, and 190 sudden odometer jumps. The voltage review threshold was not claimed to be a manufacturer validity limit. These flags can overlap, so their counts must not be added as if each row belonged to one exclusive category.

The preprocessed file also disagreed with source-derived formulas in some places: 30,057 `Instant_kW` values differed from voltage × current under the checked relationship, and 1,395 reported average pack temperatures differed from the mean of available component temperatures. Horsepower remained consistent with `Instant_kW`. These comparisons show that the researcher's processing changed or retained values in ways that the public files do not fully document.

4.5 Temporal behavior

Timestamps parse as Unix microseconds and are globally ordered. The median positive interval within a trip is 0.25 seconds, consistent with nominal quarter-second logging. Sampling is irregular: the raw data has 15,825 within-trip gaps longer than one second, including 358 longer than 60 seconds. The preprocessed data has 7,984 gaps over one second.

This matters because row count is not the same as elapsed time. Four rows usually cover about one second, but not always. Sequence construction and alert grouping therefore use actual timestamps and stop at trip boundaries or unsafe gaps.

4.6 Leakage risks found during EDA

Data leakage occurs when training receives information it would not legitimately have at prediction time, or when evaluation rows are too closely related to training rows. The major risks were:

- random row splitting would place neighboring quarter-second observations into different splits;
- windows could overlap across train and test boundaries;
- raw and preprocessed versions overlap and cannot be treated as independent sources;
- derived power or average-temperature columns can encode other measurements directly;
- trip IDs and timestamps can reveal recording period;
- SOH, odometer, and time follow the same single-vehicle aging trajectory.

These findings shaped the preparation pipeline: whole-trip chronological splits, source lineage, segment boundaries, training-only scaling, and conservative duplicate-vector screening.

5. Data Preparation and Preprocessing

5.1 EDA versus preparation

EDA describes and questions the data. Preparation applies a reproducible policy to create the exact inputs an experiment will use. For example, EDA discovers that drivetrain −1 values are common. Preparation decides whether that pattern blocks a particular feature view, records the reason, and exports retained rows without modifying the original CSV.

The first preparation run, `d7e9e32e1ff2981a`, used six features: Speed, Torque, Motor Temperature, HV Battery Voltage, Pack 1 Temperature, and SOC. Its eligibility mask excluded many periods with unavailable drivetrain values. The negative-speed review showed that useful battery measurements could remain available during those periods. The pipeline was therefore extended, not rewritten, with a separately versioned battery-only view.

5.2 Final battery-only inputs

Preparation run `1d6fc5b045ee25d0` uses exactly these model features, in this order:

1. `HV_Bat_Voltage` in volts;
2. `Pack_1_C_Temp` in degrees Celsius;

3. `SOC` in percent.

Speed, RPM, torque, motor temperature, timestamps, trip IDs, source-row IDs, quality flags, and split names remain useful metadata but do not enter the three-column model matrix. Unavailable drivetrain telemetry no longer automatically rejects a battery observation.

5.3 Quality policy

The battery view uses the raw CSV as its source. It does not impute measurements, clip outliers, convert uncertain units, or overwrite source values. Battery-specific blockers cover missing/non-finite inputs, out-of-domain SOC, documented temperature-sentinel bands, and other explicit eligibility rules. Suspicious measurements remain visible in an auditable row ledger with bit-coded quality flags.

The battery audit retained 2,316,127 of 2,322,105 raw rows as battery-quality eligible before split and cross-split-repeat policies. It excluded 5,978 under battery-specific rules. This is much less aggressive than rejecting every row with any general quality flag, because tire-pressure or drivetrain availability should not automatically invalidate voltage, battery temperature, and SOC.

5.4 Chronological splitting

The parent split assigns whole trips in chronological order and keeps an embargo of about 90.5 seconds, based on a 30-second history, 60-second forecast horizon, and 0.5-second tolerance. The battery view reuses those memberships exactly:

Split	Source trips	Retained trips	Feature rows	Recorded days retained
Train	142	139	1,623,891	74
Validation	34	34	351,087	19
Test	51	50	340,986	25

No trip appears in more than one split. A conservative rounded-feature-vector screen withheld 74 validation and 89 test rows that repeated an earlier split's vector. The analysis describes this as a screen, not proof that all remaining observations are independent.

Why avoid a random row split? If row 1000 enters training and row 1001 enters test, the model is tested on a state recorded roughly 0.25 seconds after something it already saw. That can produce impressive numbers without measuring future generalization.

5.5 Robust scaling

Features use robust scaling fitted on retained training rows only:

$$
x_{scaled} = \frac{x - \text{training median}}{\text{training IQR}}
$$

Feature	Training median	Training IQR
HV battery voltage	379.59 V	33.67 V

Pack 1 temperature	24.30 °C	14.86 °C
SOC	81.7796%	26.39555 percentage points

The median centers a typical training value near zero. The interquartile range (IQR) scales by the spread of the middle half of the training distribution. This is less sensitive to extreme values than mean/standard-deviation scaling.

The same frozen numbers transform validation, test, and later inference inputs. Recomputing scaling on test data would leak information about the held-out distribution and make deployed behavior inconsistent.

5.6 Saved preparation artifacts

The preparation directory stores:

- scaled feature Parquet files for train, validation, and test;
- matching metadata Parquet files containing source rows, timestamps, trips, segments, flags, and lineage;
- the complete row ledger and segment table;
- frozen trip splits;
- `transformer.json` with feature order, medians, IQRs, and fit scope;
- configuration, flag dictionary, checksums, and a manifest;
- sequence-index tables for the conditional battery-temperature experiment.

The sequence tables contain 37,753 training, 6,595 validation, and 4,514 test candidates for a 30-second history and roughly 60-second target horizon. They store indices and provenance rather than duplicating large arrays. No temperature model was trained in this work.

6. Isolation Forest Model Training

6.1 The concept before the terminology

Imagine repeatedly separating a room of people using random questions such as “Is height below 170 cm?” or “Is age below 30?” A person with a very unusual combination may become alone after only a few questions. Someone in a crowded, common group usually needs more questions to isolate.

Isolation Forest applies that idea to numerical data. Each **isolation tree** repeatedly chooses a feature and a random split between observed values. A point's path length is how many splits it takes to isolate it. Across many randomized trees, unusually easy-to-isolate points receive more unusual scores.

The “forest” is an ensemble of trees. Combining many randomized views is more stable than trusting one random tree.

6.2 What training means here

For a supervised classifier, training adjusts parameters to match known answers. Isolation Forest has no fault labels. Training builds randomized isolation trees from the distribution of the three scaled battery features. The model learns the geometry and density of observed training combinations, not the meaning of a battery failure.

All 1,623,891 retained training observations were supplied to `.fit()`. They represented 139 retained trips and 74 recorded dates. The implementation never fitted on validation or test rows.

A subtle parameter detail is useful: the forest was fitted using all training observations at the estimator level, while `max_samples="auto"` means each individual tree samples at most 256 observations under scikit-learn's behavior. Many trees and randomized subsets provide multiple views of the large training set.

6.3 Exact configuration

Parameter	Value	Purpose
`n_estimators`	200	Build 200 isolation trees for a more stable ensemble.
`max_samples`	`auto`	Let scikit-learn use its standard per-tree sample rule.
`max_features`	1.0	Make all three input features available to each estimator.
`bootstrap`	`false`	Sample without bootstrap replacement.
`random_state`	42	Make randomized construction reproducible.
`n_jobs`	−1	Use available CPU cores where supported.
`contamination`	`auto`	Keep the estimator's internal offset behavior; the project uses its own explicit alert budget for demo flags.

The environment recorded Python 3.12.13, scikit-learn 1.9.0, NumPy 2.5.1, pandas 2.3.3, PyArrow 25.0.0, and joblib 1.5.3.

6.4 Training, scoring, prediction, and thresholding

These are different operations:

1. **Training / fitting:** build the isolation trees using training inputs.
2. **Scoring:** call `score_samples(X)` to obtain a continuous value for every row.
3. **Thresholding:** compare each saved score with a chosen cutoff.
4. **Prediction:** produce a category such as typical/unusual from the comparison.

The project kept scoring and thresholding explicit. Its rule is:

```
unusual = score_samples < -0.6676090615628322
```

It did not interpret the score as a probability and did not rely on `predict()` to claim a true fault class.

6.5 What was saved

The experiment directory contains `model.joblib`, configuration and provenance metadata, the frozen protocol, development/test summaries, threshold candidates, scores and intervals for each split, fit-sample identifiers, and checksum records. The fitted model is 2,980,717 bytes with SHA-256 `93c7ef791fea…c2ef4cef`.

Training took 1.770 seconds in the recorded local run. That runtime is evidence about one environment, not a guaranteed benchmark for every computer. Reproducibility checks rebuilt the same-seed model byte-for-byte and matched scores exactly on a 4,096-row validation probe. Cross-seed robustness was not established.

7. Anomaly Scores and Threshold Selection

7.1 Understanding `score_samples()`

`score_samples(X)` returns one continuous score per observation. In this fitted model, lower values mean more statistically unusual feature combinations. A score of −0.69 is therefore more unusual than −0.48 according to this model.

The numerical scale is model-specific. It is not “69% risky,” a temperature severity, or a probability of mechanical failure. Scores are meaningful only with the exact saved model, frozen preprocessing, feature order, and experiment protocol.

Scikit-learn also provides `decision_function(X)`, related to the score and the estimator's internal offset. Negative `decision_function` values follow that configured offset. DrivePulse AI's demo classification instead uses the explicitly saved `score_samples` threshold, so the two ideas must not be confused.

7.2 How the threshold was selected

With no verified fault labels, there is no honest way to optimize a threshold for accuracy, recall, F1, or F2. The project evaluated predefined training alert budgets of 0.1%, 0.5%, and 1.0%. These percentages mean “how many training observations can enter the review queue,” not “estimated fault prevalence.”

The middle budget, 0.5%, was frozen as a provisional operating policy. Its training-score quantile produced the exact threshold:

> **−0.6676090615628322**

The comparison is strict: a score lower than the threshold is flagged; a score exactly equal to it is not. The threshold was selected using training and validation behavior before the one frozen test evaluation. The frozen protocol explicitly says that no labels or operator-capacity evidence justify calling it optimal.

7.3 Why freeze a threshold?

Freezing means writing the rule into an immutable experiment record before looking at final test behavior. If the team changed the threshold after seeing too many test alerts, the test set would become part of model selection. Its result would no longer represent a held-out check.

The continuous score remains the primary model output. The flag is a presentation and review policy layered on top.

7.4 From row flags to intervals

Quarter-second telemetry can produce hundreds of adjacent flagged rows during one unusual period. Showing each as a separate alert would overwhelm a reviewer. The aggregation code groups rows only when they:

- are all flagged by the frozen rule;
- are consecutive source observations;
- belong to the same trip and safe recording segment;
- have positive timestamp gaps no larger than one second.

A grouped interval is called **persistent** when its observed duration is at least two seconds. This two-second value is a provisional review filter, not a validated physical fault duration.

An individual unusual row may be a brief transition or measurement issue. A persistent interval shows that an unusual combination continued across consecutive observations. Persistence makes the pattern easier to review, but still does not establish a fault.

8. Model Evaluation and Validation

8.1 What can be evaluated without labels?

Without confirmed fault labels, the project cannot calculate fault-classification accuracy, precision, recall, F1, F2, or ROC-AUC. It can evaluate engineering and statistical behavior:

- whether train, validation, and test provenance is separate;
- score distributions and alert rates;
- concentration by trip and date;
- persistence and recorded-hour frequency;
- interaction with quality flags;
- deterministic model and output reproduction;
- whether manually reviewed intervals look coherent or artifact-prone.

This evaluates a baseline as an **unusual-pattern detector**, not as a battery-fault detector.

8.2 Verified results

Metric	Train	Validation	Test
Scored observations	1,623,891	351,087	340,986
Flagged observations	8,109	1,857	14,041
Flagged percentage	0.499%	0.529%	4.118%
All candidate intervals	127	114	317
Persistent intervals	60	49	152
Point/short intervals	67	65	165
Recorded hours	121.44	25.23	26.79
Persistent alerts/hour	0.494	1.942	5.674
Trips scored	139	34	50
Trips with persistent alerts	3	2	5

Training and validation flag rates are close to the chosen half-percent budget. Test is much higher. That does not automatically mean the test period has more faults. It means later observations more often fall into regions that the model considers unusual relative to training.

8.3 Why the test alert rate was higher

Saved analysis found a clear distribution shift and concentration:

- median battery temperature was 24.3 °C in training and 35.7 °C in test;
- test contained 37,313 rows above the training maximum temperature of about 43.1 °C;
- 5,736 flagged test rows were in that high-temperature group;
- almost all test flags were concentrated in Trips 231, 219, 200, 229, and 201; Trip 198 contributed only five short/point flags.

Temperature alone is not a complete explanation. Validation contained 25,893 rows above the same training maximum with no flags among those rows. Isolation Forest scores combinations of voltage, temperature, and SOC, so context in the three-dimensional feature space matters.

Trip 231 included a reviewed interval with temperature logged near 55.3 °C, voltage about 329.82–340.24 V, and SOC about 25.80–25.98%. Its speed, RPM, and torque were also −1 during the reviewed period. A nearly constant high temperature plus unavailable drivetrain signals could represent a valid operating period, stale values, or acquisition trouble. The repository correctly leaves the cause unresolved.

8.4 What the evaluation establishes

The experiment establishes that:

- the frozen model reproducibly scores the prepared rows;
- the threshold and aggregation policy produce a manageable validation review set;
- later data can shift enough to raise alerts substantially;
- alerts often occur in sustained, trip-concentrated patterns;
- provenance permits a reviewer to trace a score back to original source rows.

It does not establish sensitivity to real failures, a false-alarm rate against verified healthy operation, generalization to other vehicles, or a safe operational threshold. The test period had already been inspected during data-quality work, so it is an exploratory held-out evaluation rather than completely untouched independent confirmation.

9. Problems Encountered and How We Addressed Them

This chapter records only problems supported by repository evidence. It separates a mitigation from a complete resolution.

9.1 No confirmed fault labels

Problem. The files contain no verified healthy/faulty label, fault category, linked DTC outcome, repair event, or failure timestamp.

Discovery. EDA, the column dictionary, parameter workbook, and ML-readiness inventory were searched for label and historical-outcome fields.

Why it mattered. A supervised fault classifier needs confirmed answers for both training and evaluation. Without them, a rare value cannot be called a fault and a normal-looking row cannot be certified healthy.

Response. The project chose unsupervised anomaly detection and uses conservative language. It did not create artificial fault labels. Reports avoid classification metrics that require ground truth.

Status. Unresolved dataset limitation. Real fault validation requires expert-adjudicated events and independent vehicles.

9.2 Negative drivetrain values

Problem. Speed, RPM, and torque frequently equal −1, especially in the test period.

Discovery. A dedicated negative-speed review counted values and checked gear, co-occurring fields, trips, splits, and retained battery signals.

Why it mattered. The original six-feature view required drivetrain measurements. That rejected useful battery rows when drivetrain polling appeared unavailable. It also reduced test coverage much more than train/validation coverage.

Response. The original artifacts were preserved. A new versioned battery-only view requires voltage, Pack 1 temperature, and SOC while retaining a drivetrain-availability indicator as metadata. It never converts −1 to zero or absolute speed.

Status. Mitigated for the battery experiment. The exact −1 convention remains undocumented, so the acquisition interpretation is still a hypothesis.

9.3 Missing, sentinel, and suspicious measurements

Problem. Raw data contains missing SOH, SOH zero values, pack-temperature sentinel-band values, high-voltage review values, tire-pressure zeros, and odometer discontinuities.

Discovery. EDA and the quality audit counted domain violations, non-finite values, suspicious ranges, and temporal inconsistencies.

Why it mattered. Silent imputation or blanket outlier removal could invent clean behavior or erase precisely the unusual patterns an anomaly experiment should review.

Response. Source files remain immutable. The pipeline records overlapping bit flags and explicit eligibility reasons. It excludes only rows that violate the selected battery view's documented requirements; unrelated tire or drivetrain flags do not block a battery row.

Status. Auditable, not fully resolved. Some ranges and units lack authoritative validity limits.

9.4 Raw versus preprocessed inconsistency

Problem. Raw and preprocessed records overlap, while processed power and temperature-derived values sometimes disagree with source formulas.

Discovery. Exact timestamp/index comparisons and numerical formula checks found 30,057 processed power mismatches and 1,395 average-temperature mismatches.

Why it mattered. Treating versions as independent would duplicate evidence. Depending on altered derived fields could introduce undocumented assumptions.

Response. The battery preparation uses the raw file only. The processed version remains comparison evidence. The experiment does not pool the two versions or use the disputed processed derived fields.

Status. Experiment risk controlled. The researchers' complete preprocessing algorithm was not published, so all transformations cannot be reconstructed.

9.5 Time-series leakage

Problem. Consecutive rows are often only 0.25 seconds apart. Random row splitting would place nearly identical temporal neighbors in training and evaluation.

Discovery. Timestamp distributions, adjacent similarity, trip blocks, and gap analysis showed strong temporal dependence.

Why it mattered. A model could appear to generalize while seeing almost the same operating state on both sides of a split.

Response. Whole trips were assigned chronologically, boundaries preserve an embargo, windows cannot cross trips or gaps, repeat vectors are screened across splits, and scaling fits training only.

Status. Controlled within this dataset. All periods still come from one vehicle, so vehicle-specific leakage and limited generalization remain conceptual limits.

9.6 Distribution shift in the test period

Problem. Test flags rose to 4.118%, compared with roughly 0.5% in train and validation.

Discovery. Score, feature, trip, date, temperature, and quality-flag analyses showed warmer later conditions and concentration in a small set of trips.

Why it mattered. A fixed threshold may create many alerts when operating conditions differ from training. The increase could be environment, acquisition behavior, vehicle change, or faults; the dataset cannot adjudicate the cause.

Response. The threshold was not changed after test inspection. The report documents the shift, preserves continuous scores, and treats test evaluation as exploratory.

Status. Unresolved scientific limitation. More vehicles, seasons, operating contexts, and expert labels are needed.

9.7 Score and threshold interpretation

Problem. A numeric anomaly score can easily be mistaken for a probability, and a threshold can be mistaken for a known failure boundary.

Discovery. This was an interpretation risk inherent in unlabeled anomaly detection and explicitly addressed in experiment and mobile requirements.

Response. The protocol calls the threshold provisional and operational. The app says “statistically unusual,” presents the raw score and threshold, and explains that the observation is not a confirmed fault.

Status. Mitigated in documentation and UI. User studies and expert review have not validated comprehension.

9.8 Reproducibility and deployment separation

Problem. A notebook-only experiment can become impossible to reproduce or integrate.

Response. The project saved configurations, input and output hashes, a joblib model, frozen transformer, source-row metadata, frozen protocol, reports, and reusable Python modules. The fixture exporter aligns saved scores by stable source rows rather than approximate timestamp joins.

Status. Reproducible locally according to recorded checks. Runtime FastAPI inference and comparison against saved scores remain planned.

10. Selection of the Two Demo Recordings

10.1 Why use two clips?

A demo needs a short, understandable example without inventing data. One clip shows what ordinary model behavior looks like; the other shows a real interval that crossed the frozen threshold. Together they explain that the system has both a continuous telemetry view and an experimental review signal.

The clips are not video files. They are ordered JSON samples with original timestamps. The replay advances through those samples using the difference between actual timestamp offsets.

10.2 Verified clip summary

Property	Clip A—Typical telemetry	Clip B—Unusual pattern
Clip ID	leaf-typical	leaf-unusual
Split / trip	Validation / 153	Validation / 167
Inclusive source rows	1,637,821–1,638,031	1,759,260–1,759,464
Observations	211	205
Recorded duration	52.500 s	51.001 s
UTC start	2022-03-17 07:39:06.251	2022-04-05 05:32:33.000
UTC end	2022-03-17 07:39:58.751	2022-04-05 05:33:24.001
Flagged rows	0	132
Persistent intervals	0	2
Quality flags	0 throughout	0 throughout

Clip A ranges from 332.12–354.07 V, 22.8–23.6 °C, and 61.1817–62.4351% SOC. Its saved scores range from approximately −0.582106 to −0.475257, all above the frozen unusual threshold.

Clip B ranges from 316.95–340.50 V, 13.9–14.1 °C, and 37.9172–39.2762% SOC. Its scores range from approximately −0.688109 to −0.633844. Two groups remain below the threshold long enough to meet the persistent rule:

Interval	Source rows	Rows	Duration

1	1,759,328–1,759,436	109	26.999 s
2	1,759,438–1,759,455	18	4.251 s

10.3 How the clips were selected and verified

The selection process reviewed typical, persistent, short, high-temperature, low-SOC, and possible-artifact candidates. Each accepted clip had to remain inside one trip and recording segment, maintain source-row order, have a saved score for every row, use the exact threshold, and contain no gap above one second.

Clip A provides measurement movement without an alert. Clip B provides context before, during, and after two saved persistent intervals. Both come from validation, so the demo does not need to present a problematic test trip as its main example.

The high-temperature Trip 231 test example was not chosen. Its unusual temperature and unavailable drivetrain fields created unresolved acquisition questions that could make a short demonstration misleading. Rejecting that visually dramatic example was a good scientific decision: clarity should not come from overstating uncertain data.

10.4 Why these are genuine results

The exporter reads the raw source lineage, preparation metadata, saved scores, and saved interval files. It checks source-row identity, values, ordering, trip/segment membership, quality flags, and artifact hashes. It does not load the model, retrain it, change scores, or inject unusual values.

The resulting 416 samples occupy about 102 KB. Original timestamps, source rows, volts, degrees Celsius, SOC percent, scores, threshold flags, and persistent boundaries are preserved. Playback offset is derived from the original timestamp relative to the clip start.

11. How ML Results Reached the Mobile App

11.1 The current data path

Official raw Nissan Leaf CSV │ ▼ Battery-only preparation run 1d6fc5b045ee25d0 • validate and flag rows • preserve source lineage • chronological trip splits • training-only robust scaling │ ▼ Isolation Forest experiment 0c777345c64dfeba • fit on training split • score train/validation/test offline • apply frozen threshold • aggregate persistent intervals │ ▼ Saved Parquet/CSV/JSON experiment artifacts │ ▼ Bounded exporter verifies and selects two clips │ ▼ mobile/src/demo/recorded/clips.json │ ▼ Local typed provider → replay store → React Native screens

Everything through score generation happened offline in Python. The mobile app reads saved outcomes. The Isolation Forest and scikit-learn are not bundled into the Android application.

11.2 What `clips.json` stores

At the clip level, the fixture stores a schema version, clip identity and title, source dataset, vehicle description, preparation and experiment IDs, trip/split/segment, UTC range, duration, sample count, frozen threshold, persistent intervals, and provenance hashes.

Each sample stores:

- stable `source_row`;
- original UTC timestamp;
- playback offset in milliseconds;
- high-voltage battery voltage in volts;
- Pack 1 temperature in °C;
- SOC in percent;
- saved Isolation Forest `score_samples` value;
- the strict threshold result;
- original quality flags.

The file contains recorded measurements and model-derived scores. Clip titles and explanatory text are presentation metadata. It contains no invented health percentage, severity, range estimate, failure probability, remaining life, or maintenance recommendation.

11.3 How measurements and scores stay aligned

Approximate timestamp joins are risky because clocks can be rounded and samples can arrive close together. The exporter instead uses stable source-row identifiers and validates the associated trip, segment, timestamp, and feature values. The mobile validator checks uniqueness, ordering, threshold semantics, clip endpoints, and interval boundaries again when loading.

11.4 Replay versus live inference

Saved-result replay means the app presents a small sequence whose measurements and scores were computed earlier. It is deterministic and works without a backend or database after the fixture is bundled.

Live inference would receive a new telemetry observation, apply the same frozen transform, load `model.joblib`, calculate a new score, apply the protocol, and return a result. That service does not currently exist in FastAPI.

The replay is still a legitimate integration artifact: it demonstrates source-to-screen provenance and user experience. It must be introduced as recorded research telemetry, never as a live connection to a vehicle.

12. Step 2A: Mobile Architecture and Implementation

12.1 Technology choices

The mobile application uses React Native with Expo, TypeScript, and Expo Router. React Native renders native Android/iOS components from JavaScript/TypeScript. Expo supplies the project toolchain and native-module integration. Expo Router maps files to navigation routes. TypeScript adds compile-time contracts for data objects and component inputs.

The inspected versions include Expo 57.0.22, React Native 0.86.3, React 19.2.3, Expo Router 57.0.21, and TypeScript 6.0.3.

12.2 Navigation and screens

The authenticated application exposes three primary tabs:

1. **Overview**—vehicle identity, recorded timestamp, current voltage, temperature, SOC, replay status, and current score interpretation.
2. **Telemetry**—timestamp-positioned histories for voltage, temperature, and SOC.
3. **Insights**—full-recording review of flagged samples and persistent intervals.

Supporting routes provide Recorded Replay, Insight Details, and Account. Older simulated-data routes remain in the repository with explicit labels but are outside the recorded-data MVP navigation. Authentication routes were preserved.

12.3 Typed contracts

`recorded-telemetry.ts` defines the JSON structure for vehicle metadata, clip metadata, samples, persistent intervals, replay state, insight summaries, and loading/error states. Names use `snake_case` so a future FastAPI response can match the TypeScript contract without a translation layer.

A simplified sample contract is:

```
type TelemetrySample = { source_row: number; original_timestamp_utc: string; playback_offset_ms: number; voltage_v: number | null; temperature_c: number | null; soc_percent: number | null; score_samples: number | null; unusual: boolean | null; quality_flags: number; };
```

Nullable values force the interface to show “Unavailable” rather than fabricate a number.

12.4 Provider abstraction

Screens do not import the fixture directly. `RecordedTelemetryProvider` exposes two asynchronous operations: list clip metadata and fetch one complete clip. The current local adapter imports `clips.json`, validates it, and returns copies so playback cannot mutate the fixture.

This boundary is important because Step 3 can provide an HTTP adapter with the same methods. The screens and replay state should not need to know whether data came from a local JSON bundle or FastAPI.

12.5 Shared replay state

`RecordedReplayProvider` holds the selected clip, catalog/loading/error state, current sample index, and playback status. It prevents stale asynchronous requests from replacing a newer selection. Selecting a clip resets to its first sample in a paused state.

Playback uses the difference between consecutive `playback_offset_ms` values. It advances exactly one observation per timer callback, so a delayed JavaScript event cannot skip an alert boundary. Pause holds the current index. Restart returns to the first row and pauses. Reaching the last sample produces an ended state. Moving the app to the background pauses playback.

Overview and Telemetry share the same current sample. Telemetry draws history only up to that position, while Insights clearly describes its interval list as a full-recording review.

12.6 SVG charts

The chart component uses the existing `react-native-svg` dependency. Horizontal placement follows timestamps rather than assuming uniform row spacing. Missing readings break a line rather than being silently connected. Interval shading and point markers connect the current observation and unusual regions to the saved experiment metadata.

12.7 Firebase preservation

Firebase continues to manage mobile login, signup, email verification, password reset, session restoration, and sign-out. Protected navigation still wraps the recorded replay provider. The recorded-data work did not add Firebase credentials or alter the native authentication configuration.

The account flow can synchronize a minimal verified profile with FastAPI/PostgreSQL when configured, but replay itself does not depend on that service. An unavailable profile backend must not erase the Firebase account or prevent local clips from loading.

12.8 Loading and errors

The provider and store represent loading, ready, empty, and error states explicitly. Unknown clips, malformed fixture data, missing values, and request failures produce visible fallback or retry behavior. The application never falls back silently to conventional-vehicle demo values while labeling them Nissan Leaf telemetry.

13. Step 2B: Professional UI Improvements

13.1 Why a visual pass was necessary

Step 2A established correct architecture and behavior. Step 2B concentrated on whether the same information could be scanned, understood, and controlled on an Android screen. Visual polish here means consistency and clarity, not decorative effects or new product claims.

13.2 Small design system

The recorded screens use a centralized token module for neutral canvas/surface colors, a restrained green accent, an amber unusual-pattern state, text hierarchy, spacing, corner radii, and minimum touch sizes. Reusable components apply the same treatment to sections, metrics, status badges, buttons, headers, and explanatory notes.

The design intentionally avoids glowing panels, excessive gradients, random accent colors, and generic health gauges. A measurement is prominent because of typography and hierarchy, not because it has an invented score.

13.3 Screen improvements

Overview now follows a clear reading order: recorded vehicle identity, provenance and time, replay status, core battery measurements, anomaly interpretation, and actions. Units are always visible, unavailable values are explicit, and the recorded-data label prevents a live-vehicle misunderstanding.

Telemetry uses responsive SVG view boxes, channel-specific numeric precision, readable UTC endpoints, current-position lines, current markers, unusual-sample markers, and persistent-interval shading. Charts remain faithful to observed timestamps and do not smooth or fabricate readings.

Replay uses distinct radio-style clip selection, recording metadata, large play/pause/restart controls, an accessible progress indicator, current timestamp, measurements, score, and completion feedback. The timing algorithm itself was unchanged.

Insights separates flagged samples from persistent intervals and repeats the scientific limitation. **Insight Details** presents interval times, duration, row count, source-row range, score range, measurement context, and provenance without offering a causal diagnosis.

Account adopts the same visual surfaces and typography while retaining sign-out, username-save retry, and profile-sync retry behavior.

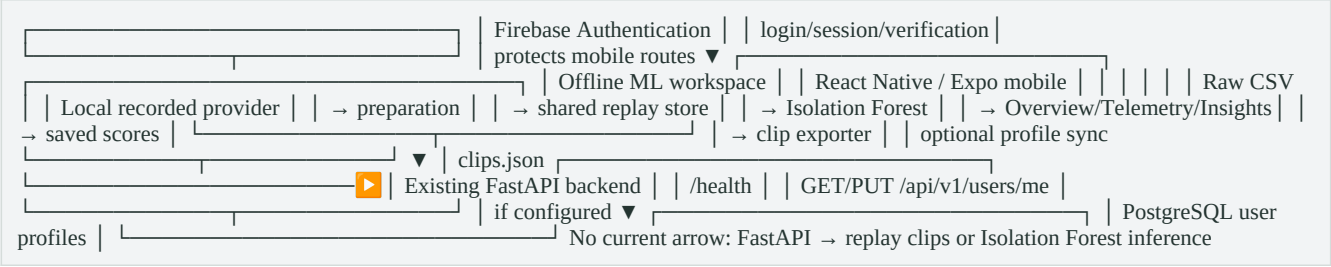
13.4 Accessibility and Android responsiveness

Controls use at least 48-pixel touch targets. Components include accessibility roles, labels, values, and state descriptions where appropriate. Charts provide text summaries for TalkBack because a visual line alone is not accessible. Scrollable layouts, safe-area spacing, long-label handling, and narrow-screen sizing were reviewed in code.

Step 2B's documented checks passed TypeScript compilation, ESLint, all 79 Jest tests across eight suites, and the Android Hermes JavaScript export. Those checks do not prove native runtime appearance. Physical Android testing is still required for font rendering, TalkBack order, landscape behavior, system navigation-bar contrast, and final spacing on the demonstration phone.

14. Current Project Architecture and Status

14.1 Current system diagram



14.2 Component status

Component	Status	Evidence and limit
Dataset acquisition and EDA	**Implemented and tested**	Manifests, checksums, reports, tables, and validation outputs exist.
Battery preparation	**Implemented and tested**	Versioned artifacts and leakage checks exist; original sources unchanged.

Isolation Forest training/scoring	**Implemented and tested**	Saved model, scores, protocol, checksums, and reproducibility checks exist.
Real-world fault diagnosis	**Not established**	No verified fault labels or independent vehicle validation.
Recorded clip export	**Implemented and tested**	Deterministic fixture and hash; source-row validation.
Mobile recorded replay	**Implemented and automated-tested**	Local provider, state, screens, charts, 79 tests, JS export.
Physical Android UI acceptance	**Not verified for Step 2B**	Requires hands-on device checks.
Firebase mobile authentication	**Implemented; reported working on demo phone**	Preserved by Step 2A/B; live server credentials not inspected here.
FastAPI health/profile API	**Implemented**	`/health`, protected GET/PUT `/api/v1/users/me`.
Replay API	**Not implemented**	Proposed contract exists only in documentation.
Runtime ML inference API	**Not implemented**	Backend does not load `model.joblib` or transformer.
PostgreSQL profile connectivity	**Not verified**	Code/migration exist; live database access was outside this work.

14.3 Where Firebase, FastAPI, and PostgreSQL fit

Firebase authenticates the mobile user. The mobile app can obtain an ID token. The current FastAPI backend is designed to verify that token with Firebase Admin and synchronize a minimal user profile into PostgreSQL. Client Firebase configuration, server Admin credentials, ID tokens, and database credentials are different things and must remain separate.

FastAPI currently serves a liveness route and two profile routes. Files named for vehicles, telemetry, predictions, and reports contain placeholders rather than functional APIs. It does not serve the recorded clips and cannot call the Isolation Forest.

PostgreSQL is useful for user profiles and future persistent product records. It is unnecessary for the present recorded telemetry demo because the fixture is bundled locally. A Step 3 replay endpoint could also serve the two immutable clip files without a database. Database connectivity was not verified during this documentation task.

14.4 What remains for Step 3

Step 3 needs a small FastAPI replay catalog/detail API, an authenticated or deliberately scoped access policy, and a mobile HTTP provider adapter. Runtime inference is a separate extension requiring exact loading of the frozen transformer, feature ordering, model artifact, score rule, and comparison against saved reference scores.

The current app is well positioned for that change because its UI consumes a provider interface rather than importing fixture data throughout the component tree.

15. Future Development

15.1 Near-term integration milestones

1. **Add FastAPI replay endpoints.** Implement `GET /replay/clips` for compact metadata and `GET /replay/clips/{clip_id}` for one validated clip. Return the existing snake-case schema and 404 for unknown IDs.
2. **Add a mobile HTTP provider.** Use the existing API base URL and Firebase bearer-token behavior, while preserving loading, error, retry, and local development options.
3. **Verify on the physical Android phone.** Test authentication, clip selection, playback controls, background pause, tab changes, interval details, TalkBack, layout, and local-network addressing.
4. **Load the frozen model in a backend inference module.** Load, do not refit, `transformer.json` and `model.joblib`; enforce exact features and types; expose model/version provenance with each score.
5. **Compare runtime scores with saved scores.** Score only the selected reference observations initially and require numerical agreement within an explicitly chosen tolerance. This tests integration without retraining.

15.2 Later scientific work

Temporal features may help distinguish a single odd row from a changing battery pattern, but any new experiment needs its own leakage-safe preparation and frozen protocol. A lightweight autoencoder could be compared later, but greater model complexity will not create missing fault labels.

Battery-temperature forecasting can use the saved sequence indices as a starting point. It remains conditional because sampling, target reliability, single-vehicle scope, and physical interpretation need careful validation.

Fault classification needs confirmed diagnoses, event timestamps, normal controls, and held-out vehicles. Remaining-useful-life estimation needs longitudinal capacity measurements, usage/charging histories, component replacements, and known end-of-life outcomes. A falling reported SOH trace from one vehicle is not enough.

15.3 A sensible learning order

For a student, the next concepts to study in this repository are:

1. trace one clip sample backward from UI JSON to source row;
2. implement and test the read-only replay API contract;
3. understand Firebase token verification separately from profile storage;
4. reproduce one frozen preprocessing calculation by hand;
5. compare one backend score with its saved score;
6. design expert-review labels before attempting performance metrics;
7. add vehicles and outcome data before making broader diagnostic claims.

This order keeps software integration and scientific validation distinct. A connected model can still be scientifically weak; a strong experiment can still lack a deployable interface. The project needs both kinds of work.

16. Important Files and Their Purposes

The following guide prioritizes files that explain architecture and decisions. Generated Parquet partitions, every plot, and every test fixture are intentionally omitted.

16.1 Dataset and metadata

Relative path	Purpose and relationship
data/nissan_leaf_2018/raw/Dataset_NissanLeaf-2021_2022.csv	Immutable official raw telemetry. Preparation reads this source; do not edit or load it casually because it is about 1.54 GB.
data/nissan_leaf_2018/processed/Dataset_Preprocessed.csv	Researcher's smaller processed version. EDA compares it with raw; the battery model does not pool it with raw.
ml/reports/nissan_leaf_2018/metadata/download_manifest.json	Records official URLs, byte sizes, SHA-256 values, acquisition status, and license finding. Start here to verify provenance.
ml/reports/nissan_leaf_2018/metadata/ Nissan_Leaf_Model2018_Data_Description.xlsx	Publisher's parameter workbook. It supports names and units but leaves some aliases and semantics uncertain.
ml/reports/nissan_leaf_2018/metadata/ parameter_dictionary_extracted.csv	Reproducible workbook extraction used by reports and feature inventory.
ml/reports/nissan_leaf_2018/EDA_REPORT.md	Full EDA narrative with measured schemas, quality findings, distributions, correlations, time behavior, and limitations.

16.2 ML readiness and preparation

Relative path	Purpose and relationship
ml/reports/nissan_leaf_2018/ML_READINESS_REPORT.md	Evaluates which ML questions the data can support and explains SOH shortcuts, label absence, leakage, and task readiness.
ml/reports/nissan_leaf_2018/NEGATIVE_SPEED_REVIEW.md	Investigates all Speed = -1 rows and motivates a separate battery-only view.
ml/notebooks/nissan_leaf_2018/config/preparation.json	Original configurable preparation policy and chronological split settings.
ml/notebooks/nissan_leaf_2018/config/battery_only.json	Battery-specific configuration: exact features, gap rules, quality policy, scaling, and conditional forecasting horizons.
ml/src/nissan_leaf_preparation/core.py	Shared low-level schema, timestamp, quality, split, and transformation logic.
ml/src/nissan_leaf_preparation/pipeline.py	Orchestrates the original preparation run and writes auditable artifacts.
ml/src/nissan_leaf_preparation/battery.py	Extends preparation with the three-feature battery-only eligibility and artifact view.
ml/notebooks/nissan_leaf_2018/scripts/prepare_data.py	Command-line entry point for reproducible preparation; it calls package code rather than keeping logic only in a notebook.
ml/reports/nissan_leaf_2018/ DATA_PREPARATION_REPORT.md	Explains initial six-feature preparation, quality flags, splits, and blocked/conditional tasks.
ml/reports/nissan_leaf_2018/ BATTERY_ONLY_PREPARATION_REPORT.md	Explains run 1d6fc5b045ee25d0, retained counts, transformer, sequence indices, and leakage validation.

16.3 ML training and evaluation

Relative path	Purpose and relationship

ml/notebooks/nissan_leaf_2018/config/isolation_forest.json	Human-readable model, threshold-budget, batching, and aggregation configuration.
ml/src/nissan_leaf_isolation/core.py	Loads exact prepared features, fits Isolation Forest, scores in batches, calculates thresholds, and groups alerts.
ml/src/nissan_leaf_isolation/pipeline.py	Runs the versioned experiment while enforcing provenance and frozen train/validation/test behavior.
ml/src/nissan_leaf_isolation/report.py	Produces experiment tables, plots, candidate intervals, and the narrative report.
ml/notebooks/nissan_leaf_2018/scripts/train_isolation_forest.py	Reproduction entry point for the already-completed baseline. Running it would train; this documentation task did not run it.
ml/reports/nissan_leaf_2018/ ISOLATION_FOREST_BASELINE_REPORT.md	Primary explanation of objective, configuration, threshold, diagnostics, test behavior, candidates, and limitations.
ml/reports/nissan_leaf_2018/ ISOLATION_FOREST_VALIDATION.md	Independent checks of score files, interval rules, hashes, and reproducibility claims.

16.4 Saved artifacts

Relative path	Purpose and relationship
ml/artifacts/nissan_leaf_2018/preparation/1d6fc5b045ee25d0/ transformer.json	Frozen training medians/IQRs and exact feature order. Runtime inference must reuse these values.
ml/artifacts/nissan_leaf_2018/preparation/1d6fc5b045ee25d0/ manifest.json	Hashes, schemas, counts, config identity, and lineage for the battery-only preparation outputs.
ml/artifacts/nissan_leaf_2018/isolation_forest/ 0c777345c64dfeba/model.joblib	Serialized fitted scikit-learn Isolation Forest. Mobile does not load this file.
ml/artifacts/nissan_leaf_2018/isolation_forest/ 0c777345c64dfeba/frozen_protocol.json	Exact threshold, strict comparison, model configuration, aggregation policy, code/input hashes, and pre-test freeze record.
ml/artifacts/nissan_leaf_2018/isolation_forest/ 0c777345c64dfeba/development.json	Development-stage diagnostics and selection evidence before frozen test scoring.
ml/artifacts/nissan_leaf_2018/isolation_forest/ 0c777345c64dfeba/scores_*.parquet	Row-aligned saved anomaly scores for each split. The clip exporter reads these rather than rescoring.
ml/artifacts/nissan_leaf_2018/isolation_forest/ 0c777345c64dfeba/intervals_*.csv	Saved grouped alert intervals used for expert review and demo interval metadata.

16.5 Recorded clip exporter and fixture

Relative path	Purpose and relationship
mobile/scripts/export-recorded-clips.py	Bounded Python exporter that joins selected source rows to saved scores and intervals, validates provenance, and emits mobile JSON.
mobile/src/demo/recorded/clips.json	Small immutable fixture containing the 416 verified samples used by the current app. It contains saved results, not a model.
mobile/RECORDED_REPLAY.md	Architectural contract, clip provenance, replay semantics, Step 2B notes, validation results, and proposed Step 3 API boundary.

16.6 Mobile contracts, provider, state, and logic

Relative path	Purpose and relationship

mobile/src/types/recorded-telemetry.ts	TypeScript interfaces that describe fixture and future FastAPI JSON shapes.
mobile/src/api/recorded-telemetry-provider.ts	Data-source interface and current local adapter. Step 3 can add an HTTP adapter without rewriting screens.
mobile/src/lib/recorded-replay.ts	Pure threshold, formatting, replay-transition, timestamp-delay, and fixture-validation logic.
mobile/src/store/recordedReplayStore.tsx	Shared asynchronous catalog/selection state and deterministic sample-by-sample playback.

16.7 Mobile UI and navigation

Relative path	Purpose and relationship
mobile/src/components/recorded/theme.ts	Step 2B design tokens for color, type, spacing, radius, and touch targets.
mobile/src/components/recorded/ui.tsx	Shared recorded-data cards, measurements, status treatments, buttons, and empty/error states.
mobile/src/components/recorded/telemetry-chart.tsx	Timestamp-positioned SVG charts, current marker, unusual points, interval shading, and accessible summaries.
mobile/src/components/recorded/screens.tsx	Composes Overview, Telemetry, Insights, Replay, Insight Details, and Account presentation from shared state/components.
mobile/src/app/(main)/(tabs)/_layout.tsx	Defines the visible Overview, Telemetry, and Insights bottom-tab navigation.
mobile/src/app/(main)/replay.tsx	Thin Expo Router entry for Recorded Replay.
mobile/src/app/(main)/recorded-insight/[id].tsx	Route that opens a saved persistent interval in context.
mobile/src/app/(main)/account.tsx	Account route using preserved Firebase behavior and Step 2B styling.

16.8 Backend

Relative path	Purpose and relationship
backend/app/main.py	Creates FastAPI, CORS, /health, profile router, database lifecycle, and database error response. It does not include telemetry or ML routes.
backend/app/api/routes/users.py	Protected GET/PUT /api/v1/users/me endpoints using verified Firebase identity and PostgreSQL.
backend/app/api/deps.py	Supplies verified identities and request-scoped database sessions.
backend/app/core/firebase.py	Initializes Firebase Admin and verifies ID tokens; server credentials are separate from Android client configuration.
backend/app/database.py	Builds the SQLAlchemy PostgreSQL engine. Connectivity depends on external configuration and was not verified here.
backend/app/api/routes/telemetry.py	Placeholder comment only; no implemented telemetry endpoint.
backend/app/api/routes/predictions.py	Placeholder comment only; no implemented model-inference endpoint.
backend/README.md	Describes current profile/auth behavior, local setup, environment roles, and the fact that vehicles, telemetry, and reports are not persisted.

Closing study note

DrivePulse AI's most valuable result so far is not a claim that it can diagnose a Nissan Leaf. It is the disciplined chain of evidence: official source files, measured EDA, explicit limitations, leakage-aware preparation, a reproducible baseline, immutable scores, source-aligned demo clips, typed mobile contracts, and honest user language.

The next engineering step is small and clear: serve the same verified clips through FastAPI and swap the provider. The next scientific step is harder: obtain independent vehicles and expert-confirmed outcomes. Keeping those two steps separate will prevent a successful software demo from being mistaken for validated vehicle diagnostics.

Document verification notes

- Project-specific counts and thresholds were checked against repository reports, manifests, and saved JSON/CSV metadata; the full multi-gigabyte source CSVs were not reread for this documentation task.
- No dataset, preparation artifact, score, model, threshold, application source, credential, database, or experiment was modified.
- Dataset licensing beyond public download and requested research citation remains unknown.
- Live PostgreSQL connectivity, Firebase Admin credentials, runtime replay endpoints, runtime ML inference, and Step 2B physical-device appearance were not verified.